

19-ICS-Modbus - Claus for Concern

Introduction

The Story



La nieve cae con fuerza sobre Wareville mientras el caos estalla en la sede de TBFC. Lo que debería ser el día de envíos más ajetreado de la temporada se ha convertido en un desastre.

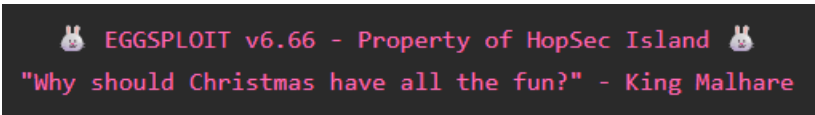
"¿Otro huevo de chocolate?!", grita un trabajador del almacén frustrado, sosteniendo otro paquete con temática de Pascua. "¿Se supone que estamos enviando regalos de Navidad!"

Los drones de reparto zumban sobre sus cabezas, con un zumbido mecánico que suena casi... burlón. Cada uno regresa de su ruta vacío, tras haber entregado su carga con éxito. Pero la carga está mal.

Te llaman al centro de mando, donde las pantallas parpadean con las estadísticas de entrega. Todo parece normal a simple vista: 1000 regalos en stock, un 98% de éxito y todos los sistemas operativos. Pero los teléfonos no paran de sonar con ciudadanos confundidos que preguntan por qué reciben huevos de chocolate en lugar de los juguetes y regalos que pidieron.

El gerente de logística abre un manifiesto de entrega. "Mira esto", dice, señalando la pantalla. "El sistema indica que entregamos un osito de peluche a la familia Miller, pero en su lugar recibieron un conejito de chocolate. Tiene el mismo peso, las mismas dimensiones, pero son artículos completamente diferentes".

Entonces, en una de las pantallas de monitoreo, un mensaje parpadea durante un segundo antes de desaparecer:



Alguien ha comprometido los sistemas de control de la flota de drones. El ataque es sofisticado: falsifica datos de sensores, manipula la selección de inventario y borra todo rastro. No es solo una broma, es un ataque calculado contra la Navidad misma.

Tu misión es clara: investigar el Sistema de Entrega de Drones TBFC, descubrir cómo el equipo Eggsplit del Rey Malhare lo ha comprometido y restaurar las entregas navideñas antes de que la Navidad se arruine.

Pero cuidado: el Rey Malhare no deja los sistemas indefensos. Hay trampas acechando al investigador descuidado. Un movimiento en falso y podrías empeorar las cosas.

Un Descubrimiento Misterioso

Al caminar por la sala de control del almacén, algo llama tu atención: un trozo de papel arrugado en el suelo, cerca de la terminal PLC. Parece que alguien lo dejó caer con prisa.

Lo recoges y lo desdoblas. La escritura es apresurada, casi frenética:

```
TBFC DRONE CONTROL - REGISTER MAP
(For maintenance use only)

HOLDING REGISTERS:
HR0: Package Type Selection
    0 = Christmas Gifts
    1 = Chocolate Eggs
    2 = Easter Baskets

HR1: Delivery Zone (1-9 normal, 10 = ocean dump!)

HR4: System Signature/Version
    Default: 100
    Current: ??? (check this!)

COILS (Boolean Flags):
C10: Inventory Verification
    True = System checks actual stock
    False = Blind operation

C11: Protection/Override
    True = Changes locked/monitored
    False = Normal operation

C12: Emergency Dump Protocol
    True = DUMP ALL INVENTORY
    False = Normal

C13: Audit Logging
    True = All changes logged
    False = No logging

C14: Christmas Restored Flag
    (Auto-set when system correct)

C15: Self-Destruct Status
    (Auto-armed on breach)

CRITICAL: Never change HR0 while C11=True!
Will trigger countdown!

- Maintenance Tech, Dec 19
```

Miras la nota, la confusión te invade. "¿Mapa de registros? ¿Bobinas? ¿Qué es todo esto?"

La terminología te resulta desconocida: HR0, C11, "Modbus" garabateado en el margen. Pero algo en ella te parece importante, como una llave que aún no sabes usar.

Guardas la nota con cuidado. "Ya veré qué significa esto más tarde", piensas. Por ahora, necesitas comprender los sistemas con los que estás tratando.

Lo que no sabes es que esta nota arrugada será precisamente lo que te salve la Navidad...

- Objetivos de aprendizaje
- Cómo los sistemas SCADA (Supervisión, Control y Adquisición de Datos) monitorizan los procesos industriales
- Qué hacen los PLC (Controladores Lógicos Programables) en la automatización
- Cómo el protocolo Modbus permite la comunicación entre dispositivos industriales
- Cómo identificar configuraciones de sistemas comprometidos en sistemas industriales
- Técnicas para remediar de forma segura sistemas de control comprometidos
- Comprender los mecanismos de protección y la lógica de trampas en entornos ICS

Answer the questions below

The clock is ticking, investigator. Christmas is hanging by a thread, and only you can pull it back from the brink.

No answer needed

SCADA (Supervisory Control and Data Acquisition)

¿Qué es SCADA?

Los sistemas SCADA son los "centros de mando" de las operaciones industriales. Actúan como el puente entre los operadores humanos y las máquinas que realizan el trabajo. Piense en SCADA como el sistema nervioso de una fábrica: detecta lo que sucede, procesa esa información y envía comandos para que las cosas sucedan.

TBFC utiliza un sistema SCADA para supervisar toda su operación de entrega con drones. Sin él, los operadores no tendrían forma de monitorear cientos de drones, gestionar el inventario ni garantizar que los paquetes lleguen a los destinos correctos. Es el orquestador invisible de la logística navideña.

Componentes de un sistema SCADA

Un sistema SCADA generalmente consta de cuatro componentes clave:

- Sensores y actuadores:** Son los ojos y las manos del sistema. Los sensores miden condiciones reales, como temperatura, presión, posición y peso. Los actuadores realizan acciones físicas: los motores giran, las válvulas se abren, los brazos robóticos se mueven. En el almacén de TBFC, los sensores detectan cuándo se coloca un paquete en la cinta transportadora y los actuadores controlan los brazos robóticos que cargan los drones.
- PLC (Controladores Lógicos Programables):** Son los cerebros que ejecutan la lógica de automatización. Leen los datos de los sensores, toman decisiones basadas en reglas programadas y envían comandos a los actuadores. Un PLC podría decidir: Si el peso del paquete coincide con el de un huevo de chocolate Y el destino es la Zona 5, cargarlo en el Dron 7. Exploraremos los PLC en detalle en la siguiente tarea.
- Sistemas de monitoreo:** Interfaces visuales como cámaras de CCTV, tableros de control y paneles de alarma donde los operadores observan los procesos físicos. El almacén de TBFC cuenta con cámaras de seguridad en el puerto 80 que muestran imágenes en tiempo real de la planta de empaque. Estos sistemas de monitoreo proporcionan retroalimentación visual inmediata: se puede observar literalmente lo que hace la automatización.

4. **Historiadores:** Bases de datos que almacenan datos operativos para su posterior análisis. Cada paquete cargado, cada dron lanzado, cada cambio en el sistema se registra. Estos datos históricos ayudan a identificar patrones, solucionar problemas y, en escenarios de respuesta a incidentes como el nuestro, reconstruir lo que hizo un atacante.

SCADA en el Sistema de Entrega con Drones

El sistema SCADA comprometido de TBFC gestiona varias funciones críticas:

1. **Selección del tipo de paquete:** El sistema decide si se cargan regalos de Navidad, huevos de chocolate o cestas de Pascua en cada dron. Esta selección se controla mediante un simple valor numérico que determina qué cinta transportadora se activa.
2. **Ruta de la zona de entrega:** Cada paquete debe llegar al barrio correcto. Las zonas 1 a 9 representan diferentes distritos de Wareville, mientras que la zona 10 está reservada para la eliminación (el océano, un sistema de seguridad para mercancías dañadas, pero también un blanco perfecto para sabotajes).
3. **Monitoreo visual:** La señal de las cámaras de CCTV proporciona observación en tiempo real del almacén. Los operadores pueden ver qué artículos se están cargando, verificar el comportamiento del sistema e identificar anomalías. Esta capa visual es crucial durante la respuesta a incidentes.
4. **Verificación de inventario:** Antes de cargar un paquete, el sistema puede comprobar si el artículo solicitado existe realmente en stock. Cuando esta verificación está deshabilitada, el sistema sigue ciegamente las órdenes, incluso si son maliciosas.
5. **Mecanismos de protección del sistema:** Funciones de seguridad diseñadas para evitar cambios no autorizados. Cuando se activan, estas protecciones monitorean modificaciones sospechosas y pueden desencadenar respuestas defensivas. Desafortunadamente, el Rey Malhare ha utilizado estas mismas protecciones como parte de su trampa.
6. **Registro de auditoría:** Cada cambio de configuración, cada inicio de sesión de operador, cada modificación del sistema debe registrarse. Los atacantes a menudo desactivan el registro para ocultar sus rastros, y eso es precisamente lo que sucedió aquí.

¿Por qué los sistemas SCADA son el objetivo?

Los sistemas de control industrial, como SCADA, se han convertido en objetivos cada vez más atractivos para ciberdelincuentes y actores estatales. He aquí el porqué:

- A menudo ejecutan software heredado con vulnerabilidades conocidas. Muchos sistemas SCADA se instalaron hace décadas y nunca se actualizaron. Los parches de seguridad existentes para el software moderno no existen para estos sistemas obsoletos.
- Las credenciales predeterminadas generalmente no se modifican. Los administradores priorizan mantener los sistemas en funcionamiento sobre cambiar las contraseñas. En entornos industriales, la mentalidad suele ser "si funciona, no lo toques", una receta para desastres de seguridad.
- Están diseñados para la confiabilidad, no para la seguridad. La mayoría de los sistemas SCADA se desarrollaron antes de que la ciberseguridad fuera una preocupación importante. Estaban pensados para redes cerradas que se consideraban seguras. La autenticación, el cifrado y los controles de acceso eran, en el mejor de los casos, una cuestión de último momento.
- Controlan procesos físicos. A diferencia de atacar un sitio web o robar datos, comprometer los sistemas SCADA tiene consecuencias reales. Los atacantes pueden provocar apagones, contaminar el suministro de agua o, en nuestro caso, sabotear las entregas navideñas.
- Suelen estar conectados a redes corporativas. El mito de los sistemas industriales "aislados" es en gran parte una ficción. La mayoría de los sistemas SCADA se conectan a redes empresariales para la generación de informes, la gestión remota y la integración de datos. Esta conectividad proporciona a los atacantes puntos de entrada.
- Protocolos como Modbus carecen de autenticación. Muchos protocolos industriales se diseñaron para entornos de confianza. Cualquiera que acceda al puerto Modbus (502) puede leer y escribir valores sin necesidad de demostrar su identidad.

A principios de 2024, se descubrió el primer malware ICS/OT, [FrostyGoop](#). El malware puede interactuar directamente con los sistemas de control industrial mediante el protocolo Modbus TCP, lo que permite lecturas y escrituras arbitrarias en los registros de los dispositivos a través del puerto TCP 502.

King Malhare ha utilizado estas mismas tácticas, no para provocar apagones, sino para sabotear las entregas navideñas manipulando directamente el sistema de control mediante el protocolo Modbus. En la siguiente tarea, exploraremos el PLC, el componente que ha comprometido.

Answer the questions below

What port is commonly used by Modbus TCP?

respuesta: 502

PLC & Modbus Protocol

¿Qué es un PLC?

Un PLC (Controlador Lógico Programable) es una computadora industrial diseñada para controlar maquinaria y procesos en entornos reales. A diferencia de su computadora portátil o teléfono inteligente, los PLC son máquinas diseñadas específicamente para brindar una confiabilidad extrema y condiciones adversas.

Los PLC están diseñados para:

- **Soportar entornos hostiles:** Funcionan a la perfección en temperaturas extremas, vibración constante, polvo, humedad e interferencias electromagnéticas. Un PLC que controla la robótica de un almacén puede soportar temperaturas gélidas en áreas de almacenamiento invernal y un calor abrasador cerca de la maquinaria de empaquetado.
- **Funcionan continuamente sin fallas:** Los PLC funcionan 24/7 durante años, a veces décadas, sin reiniciarse. Las instalaciones industriales no pueden permitirse tiempos de inactividad para actualizaciones de software o reinicios del sistema. Cuando un PLC comienza a funcionar, se espera que siga funcionando indefinidamente.
- **Ejecutan la lógica de control en tiempo real:** Los PLC responden a las entradas de los sensores en milisegundos. Cuando un paquete llega al final de una cinta transportadora, el PLC debe activar instantáneamente el brazo robótico para atraparlo. Estos requisitos de sincronización son cruciales para la seguridad y la eficiencia.
- **Interfaz directa con hardware físico:** los PLC se conectan directamente a sensores (que miden temperatura, presión, posición y peso) y actuadores (motores, válvulas, interruptores, brazos robóticos). Hablan el lenguaje eléctrico de la maquinaria industrial.

¿Qué es Modbus?

Modbus es el protocolo de comunicación que utilizan los dispositivos industriales para comunicarse entre sí. Creado en 1979 por Modicon (ahora Schneider Electric), es uno de los protocolos industriales más antiguos y ampliamente implementados del mundo. Su longevidad no se debe a sus sofisticadas funciones, sino todo lo contrario. Modbus triunfó porque es simple, confiable y funciona con casi cualquier dispositivo.

Piense en Modbus como una conversación básica de solicitud-respuesta:

- Cliente (su computadora): "PLC, ¿cuál es el valor actual del registro 0?"
- Servidor (el PLC): "El registro 0 actualmente tiene el valor 1".

Esta simplicidad facilita la implementación y depuración de Modbus, pero también significa que la seguridad nunca fue un factor a considerar. No hay autenticación, cifrado ni verificación de autorización. Cualquier persona que tenga acceso al puerto Modbus puede leer o escribir cualquier valor. Es como dejar la casa sin llave con un cartel que diga "¡Pase, todo está accesible!".

Tipos de datos Modbus

Modbus organiza los datos en cuatro tipos distintos, cada uno con una finalidad específica en la automatización industrial:

Type	Purpose	Values	Example Use Cases
Coils	Digital outputs (on/off)	0 or 1	Motor running? Valve open? Alarm active?
Discrete Inputs	Digital inputs (on/off)	0 or 1	Button pressed? Door closed? Sensor triggered?
Holding Registers	Analogue outputs (numbers)	0-65535	Temperature setpoint, motor speed, zone selection
Input Registers	Analogue inputs (numbers)	0-65535	Current temperature, pressure reading, flow rate

La distinción entre entradas y salidas es importante. Las bobinas y los registros de retención son modificables: se pueden modificar sus valores para controlar el sistema. Las entradas discretas y los registros de entrada son de solo lectura: reflejan las mediciones del sensor que se observan, pero no se pueden modificar directamente.

En el sistema de control de drones de TBFC, encontrará:

Registros de retención que almacenan los valores de configuración:

- HR0: Selección del tipo de paquete (0=Regalos, 1=Huevos, 2=Cestas)
- HR1: Zona de entrega (1-9 para zonas normales, 10 para eliminación de emergencia)
- HR4: Firma del sistema (un identificador de versión o, en este caso, la tarjeta de visita de un atacante)

Bobinas que controlan el comportamiento del sistema:

- C10: Verificación de inventario habilitada/deshabilitada
- C11: Mecanismo de protección habilitado/deshabilitado
- C12: Protocolo de volcado de emergencia activo/inactivo
- C13: Registro de auditoría habilitado/deshabilitado
- C14: Indicador de estado de restauración de Navidad
- C15: Mecanismo de autodestrucción activado/desactivado

¿Recuerda la nota arrugada que encontró antes? Ahora tiene todo el sentido. ¡El técnico de mantenimiento estaba documentando estas direcciones Modbus exactas y sus significados!

Direccionamiento Modbus

Cada punto de datos en Modbus tiene una dirección única; imagínese el número de una casa en una calle. Al leer o escribir un valor específico, se referencia a él por su número de dirección.

Detalle importante: Las direcciones Modbus empiezan en 0, no en 1. Esta indexación a cero sorprende a muchos principiantes. Cuando la documentación menciona "Registro 0", se refiere literalmente al primer registro, no al segundo.

Ejemplos del sistema TBFC:

- Registro de retención 0 (HR0) = Selector de tipo de paquete
- Registro de retención 1 (HR1) = Zona de entrega
- Registro de retención 4 (HR4) = Firma del sistema
- Bobina 10 (C10) = Indicador de verificación de inventario
- Bobina 11 (C11) = Indicador de mecanismo de protección
- Bobina 15 (C15) = Estado de autodestrucción

Modbus TCP vs. Modbus serie

Originalmente, Modbus operaba mediante conexiones serie mediante cables RS-232 o RS-485. Los dispositivos estaban conectados físicamente en una red, y este aislamiento físico proporcionaba cierto grado de seguridad: se necesitaba acceso físico al cableado para interceptar o inyectar comandos.

Los sistemas industriales modernos utilizan Modbus TCP, que encapsula el protocolo Modbus dentro de paquetes de red TCP/IP estándar. Los servidores Modbus TCP escuchan en el puerto 502 por defecto.

Esta conectividad de red ofrece enormes ventajas: monitorización remota, integración más sencilla con los sistemas empresariales y gestión centralizada. Sin embargo, también expone estos sistemas, históricamente aislados, a ataques basados en la red.

El rey Malhare explotó precisamente esta vulnerabilidad. El puerto Modbus TCP (502) del sistema de control de drones TBFC era accesible a través de la red sin necesidad de autenticación. No necesitó acceder a las instalaciones ni manipular el cableado. Simplemente se conectó al puerto 502 y comenzó a emitir comandos como si estuviera autorizado.

El problema de seguridad

Modbus no cuenta con mecanismos de seguridad integrados:

- **Sin autenticación:** El protocolo no verifica quién realiza las solicitudes. Cualquier cliente puede conectarse y emitir comandos.
- **Sin cifrado:** Toda la comunicación se realiza en texto plano. Cualquiera que monitoree el tráfico de red puede ver exactamente qué valores se leen o escriben.
- **Sin autorización:** No existe el concepto de permisos. Si puedes conectarte, puedes leer y escribir cualquier cosa.
- **Sin comprobación de integridad:** Más allá de las sumas de comprobación básicas para errores de transmisión, no existe verificación criptográfica de que los comandos no hayan sido manipulados.

Existen soluciones de seguridad modernas (VPN, firewalls, puertas de enlace de seguridad Modbus), pero son complementos, no forman parte del protocolo en sí. Muchas instalaciones industriales no han implementado estas protecciones, ya sea por cuestiones de coste, problemas de compatibilidad con equipos antiguos o

simplemente por desconocimiento.

Conectando los puntos

Ahora entiendes por qué la interfaz web de OpenPLC no mostró nada útil. King Malhare la ignoró por completo. Se conectó directamente al puerto TCP Modbus y manipuló los registros y bobinas que controlan el comportamiento del sistema.

¿Esa nota que encontraste? El técnico de mantenimiento debió descubrir lo que estaba sucediendo y comenzó a documentar los valores comprometidos. La advertencia al final —"¡Nunca cambies HR0 mientras C11 sea Verdadero!"— sugiere que descubrieron el mecanismo de la trampa antes de ser interrumpidos.

En la siguiente tarea, usaremos Python y la biblioteca `pymodbus` para investigar el sistema exactamente como lo atacó el Rey Malhare: leyendo y escribiendo directamente valores Modbus. Es hora de ver qué cambió realmente.

Answer the questions below

Now that you understand how the system works, the mission is yours, hack it back and save Christmas!

No answer needed

Practical

Ahora que comprende los conceptos básicos de ICS y Modbus, analizaremos el sistema de control de drones TBFC comprometido y aprenderemos a restaurarlo de forma segura.

Reconocimiento inicial

Como en cualquier escenario de respuesta a incidentes, comenzamos con el reconocimiento. Descubramos qué servicios se están ejecutando en el sistema objetivo.

Desde la terminal de AttackBox, ejecute un escaneo de Nmap:

```

root@tryhackme:~# nmap -sV -p 22,80,502 MACHINE_IP
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-24 10:33 -05
Nmap scan report for 10.201.65.50
Host is up (0.43s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.11 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     Werkzeug httpd 3.1.3 (Python 3.12.3)
502/tcp    open  modbus   Modbus TCP
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 48.26 seconds
```

Nota: Se especificaron los puertos 22, 80 y 502 para acelerar el análisis. Si desea realizar un análisis completo de todos los puertos, puede usar:

```

root@tryhackme:~# nmap -sV -T4 -p- -vv MACHINE_IP
```

Sin embargo, esto tardará bastante más (varios minutos), ya que escanea los 65.535 puertos.

Resultados clave:

- Puerto 80: Servicio HTTP (la señal de la cámara CCTV)
- Puerto 502: Modbus TCP (el protocolo de comunicación del PLC)

Esto es típico de la configuración de un sistema de control industrial: una interfaz web para la monitorización y Modbus para el control programático.

Confirmación visual: La señal de CCTV

Antes de profundizar en los detalles técnicos, veamos qué está sucediendo físicamente en el almacén.

Visite `http://MACHINE_IP` en su navegador.

A través de la cámara de seguridad, puede ver el almacén en tiempo real. Los brazos robóticos están trabajando a toda máquina y las cintas transportadoras funcionan a la perfección. Pero algo anda mal: en lugar de regalos de Navidad, ve:

- Huevos de chocolate de color pastel clasificándose
- Empaques con temática de Pascua en la línea de montaje
- Drones de reparto cargando huevos en lugar de regalos

El indicador de estado en la esquina muestra: Comprometido

Esta confirmación visual le indica que el problema es real y está activo. El sistema no está roto; funciona perfectamente, solo entrega los artículos incorrectos. Esta es una señal clásica de un ataque de manipulación lógica, más que de un fallo del sistema.

Mantenga esta señal de CCTV abierta en una pestaña aparte. Se actualizará a medida que realice cambios en el sistema, lo que le proporcionará información en tiempo real sobre sus esfuerzos de remediación.

Reconocimiento Modbus

Necesitaremos interrogar directamente al servidor Modbus. Aquí es donde Python y la biblioteca PyModbus se vuelven esenciales.

¿Recuerdas esa nota arrugada que encontraste antes? Sácala ahora. La terminología que te parecía desconocida (HR0, C11, "Modbus") está a punto de cobrar sentido.

Nota: Los pasos del 1 al 5 son opcionales; puedes leerlos.

Paso 1: Instalar PyModbus

En AttackBox, ya lo tenemos preinstalado, pero si usas tu propia máquina, asegúrate de tener instalada la biblioteca necesaria:

```
Terminal

root@tryhackme:~# pip3 install pymodbus==3.6.8

Collecting pymodbus
  Downloading pymodbus-3.6.8-py3-none-any.whl (231 kB)
Successfully installed pymodbus-3.6.8
```

Paso 2: Establecer la conexión

Conectémonos a la interfaz Modbus del PLC. Abramos un intérprete de Python:

```
Terminal

root@tryhackme:~# python3
Python 3.10.12 (main, Nov 20 2023, 15:14:05)
Type "help", "copyright", "credits" or "license" for more information.
>>> from pymodbus.client import ModbusTcpClient
>>>
>>> # Connect to the PLC on port 502
>>> client = ModbusTcpClient('MACHINE_IP', port=502)
>>>
>>> # Establish connection
>>> if client.connect():
...     print("Connected to PLC successfully")
... else:
...     print("Connection failed")
...
Connected to PLC successfully
>>>
```

¡Excelente! Tenemos una conexión a la interfaz Modbus del PLC. Observe que no se requirió autenticación; esta es una vulnerabilidad crítica de seguridad en el protocolo Modbus.

Paso 3: Lectura de los registros de retención

Los registros de retención almacenan valores numéricos de configuración. Según la nota, HR0 controla la selección del tipo de paquete. Veamos:

```
Terminal

>>> # Read holding register 0 (Package Type)
>>> result = client.read_holding_registers(address=0, count=1, slave=1)
>>>
>>> if not result.isError():
...     package_type = result.registers[0]
...     print(f"HR0 (Package Type): {package_type}")
...     if package_type == 0:
...         print("  Christmas Presents")
...     elif package_type == 1:
...         print("  Chocolate Eggs")
...     elif package_type == 2:
...         print("  Easter Baskets")
...
HR0 (Package Type): 1
  Chocolate Eggs
>>>
```

¡Ahí está! HR0 está configurado en 1, lo que significa que el sistema está configurado para cargar huevos de chocolate. La nota era correcta: esto es precisamente lo que causa el problema.

Revisemos HR1 (Zona de Entrega):

```
Terminal

>>> # Read holding register 1 (Delivery Zone)
>>> result = client.read_holding_registers(address=1, count=1, slave=1)
>>>
>>> if not result.isError():
...     zone = result.registers[0]
...     print(f"HR1 (Delivery Zone): {zone}")
...     if zone == 10:
...         print(" WARNING: Ocean dump zone")
...     else:
...         print(f" Normal delivery zone")
...
HR1 (Delivery Zone): 5
Normal delivery zone
>>>
```

La zona 5 es normal. Ahora revisemos HR4, la señal del sistema:

```
Terminal

>>> # Read holding register 4 (System Signature)
>>> result = client.read_holding_registers(address=4, count=1, slave=1)
>>>
>>> if not result.isError():
...     signature = result.registers[0]
...     print(f"HR4 (System Signature): {signature}")
...     if signature == 666:
...         print(" EGGSPLOIT DETECTED - King Malhare's signature")
...
HR4 (System Signature): 666
EGGSPLOIT DETECTED - King Malhare's signature
>>>
```

El valor 666 confirma que este sistema ha sido comprometido por el framework Eggsplloit. Esto coincide con el mensaje de burla que vimos anteriormente: "EGGSPLOIT v6.66".

Paso 4: Lectura de bobinas

Las bobinas son indicadores booleanos que controlan el comportamiento del sistema. La nota mencionó varias bobinas críticas. Leámoslas:

```
Terminal

>>> # Read coil 10 (Inventory Verification)
>>> result = client.read_coils(address=10, count=1, slave=1)
>>>
>>> if not result.isError():
...     verification = result.bits[0]
...     print(f"C10 (Inventory Verification): {verification}")
...     if not verification:
...         print(" DISABLED - System not checking stock")
...
C10 (Inventory Verification): False
DISABLED - System not checking stock
>>>
```

La verificación de inventario está deshabilitada. El sistema sigue órdenes ciegamente sin verificar si los artículos están disponibles. Revisemos C11:

```
Terminal

>>> # Read coil 11 (Protection/Override)
>>> result = client.read_coils(address=11, count=1, slave=1)
>>>
>>> if not result.isError():
...     protection = result.bits[0]
...     print(f"C11 (Protection/Override): {protection}")
...     if protection:
...         print(" ACTIVE - Changes are being monitored")
...
C11 (Protection/Override): True
ACTIVE - Changes are being monitored
>>>
```

¡Esto es crucial! C11 está habilitado, lo que significa que el sistema está monitoreando activamente los cambios. Recuerda la advertencia en la nota: "¡Nunca cambies HR0 mientras C11 sea verdadero! ¡Activará la cuenta regresiva!".

Comprobemos si la autodestrucción ya está activada:

```
Terminal

>>> # Read coil 15 (Self-Destruct Status)
>>> result = client.read_coils(address=15, count=1, slave=1)
>>>
>>> if not result.isError():
...     armed = result.bits[0]
...     print(f"C15 (Self-Destruct Armed): {armed}")
...     if not armed:
...         print("  Not armed yet - safe for now")
...
C15 (Self-Destruct Armed): False
  Not armed yet - safe for now
>>>
```

Buenas noticias: la autodestrucción aún no está activada. Pero lo estará si intentamos cambiar HR0 mientras C11 esté activo. Este es el mecanismo de la trampa.

Paso 5: Entendiendo la Trampa

Ahora la nota cobra sentido. El técnico de mantenimiento descubrió:

- HR0 está configurado en 1 (forzando huevos).
- La protección de C11 está habilitada (monitoreando cambios).
- Si alguien cambia HR0 mientras C11 es Verdadero, C15 se activa.
- Una vez activado C15, comienza una cuenta regresiva de 30 segundos.
- Después de 30 segundos, C12 (Descarga de Emergencia) se activa y todo se descarga a la Zona 10 (océano).

La advertencia del técnico intentaba evitar que quien encontrara esta nota empeorara las cosas.

Script de reconocimiento completo

Creemos un script completo para ver el estado completo del sistema. Salga del intérprete de Python (presione Ctrl+D o escriba "exit()") y luego cree un nuevo archivo:

```
Terminal

root@tryhackme:~# nano reconnaissance.py
```

Copia y pega el siguiente código:

```
#!/usr/bin/env python3
from pymodbus.client import ModbusTcpClient

PLC_IP = "MACHINE_IP"
PORT = 502
UNIT_ID = 1

# Connect to PLC
client = ModbusTcpClient(PLC_IP, port=PORT)

if not client.connect():
    print("Failed to connect to PLC")
    exit(1)

print("=" * 60)
print("TBFC Drone System - Reconnaissance Report")
print("=" * 60)
print()

# Read holding registers
print("HOLDING REGISTERS:")
print("-" * 60)

registers = client.read_holding_registers(address=0, count=5, slave=UNIT_ID)
if not registers.isError():
    hr0, hr1, hr2, hr3, hr4 = registers.registers

    print(f"HR0 (Package Type): {hr0}")
    print(f"  0=Christmas, 1=Eggs, 2=Baskets")
    print()

    print(f"HR1 (Delivery Zone): {hr1}")
    print(f"  1-9=Normal zones, 10=Ocean dump")
    print()

    print(f"HR4 (System Signature): {hr4}") if hr4 == 666:
    print(f" WARNING: Eggsplit signature detected")
    print()

# Read coils
print("COILS (Boolean Flags):")
print("-" * 60)

coils = client.read_coils(address=10, count=6, slave=UNIT_ID)
if not coils.isError():
    c10, c11, c12, c13, c14, c15 = coils.bits[:6]
    print(f"C10 (Inventory Verification): {c10}")
    print(f" Should be True")
```



```
print()

print(f"C11 (Protection/Override): {c11}") if c11:
print(f" ACTIVE - System monitoring for changes")
print()

print(f"C12 (Emergency Dump): {c12}") if c12:
print(f" CRITICAL: Dump protocol active")
print()

print(f"C13 (Audit Logging): {c13}")
print(f" Should be True")
print()

print(f"C14 (Christmas Restored): {c14}")
print(f" Auto-set when system is fixed")
print()

print(f"C15 (Self-Destruct Armed): {c15}")
if c15:
    print(f" DANGER: Countdown active")
print()

print("=" * 60)
print("THREAT ASSESSMENT:")
print("=" * 60)

if hr4 == 666:
    print("Eggsploit framework detected")
if c11:
    print("Protection mechanism active - trap is set")
if hr0 == 1:
    print("Package type forced to eggs")
if not c10:
    print("Inventory verification disabled")
if not c13:
    print("Audit logging disabled")

print()
print("REMEDIATION REQUIRED")
print("=" * 60)

client.close()
```

Guardar y salir (Ctrl+X, luego Y, luego Intro). Ejecutar el script:

Terminal

```
root@tryhackme:~# python3 reconnaissance.py

=====
TBFC Drone System - Reconnaissance Report
=====

HOLDING REGISTERS:
-----

HR0 (Package Type): 1
    0=Christmas, 1=Eggs, 2=Baskets

HR1 (Delivery Zone): 5
    1-9=Normal zones, 10=Ocean dump

HR4 (System Signature): 666
    WARNING: Eggsploit signature detected

COILS (Boolean Flags):
-----

C10 (Inventory Verification): False
    Should be True

C11 (Protection/Override): True
    ACTIVE - System monitoring for changes

C12 (Emergency Dump): False

C13 (Audit Logging): False
    Should be True

C14 (Christmas Restored): False
    Auto-set when system is fixed

C15 (Self-Destruct Armed): False

=====
THREAT ASSESSMENT:
=====

Eggsploit framework detected
Protection mechanism active - trap is set
Package type forced to eggs
Inventory verification disabled
Audit logging disabled

REMEDIATION REQUIRED
=====
```

Ahora tenemos una visión completa de la vulnerabilidad. Es hora de restaurar el sistema.

Remediación segura

Según nuestro reconocimiento, necesitamos:

1. Desactivar el mecanismo de protección (C11) PRIMERO
2. Cambiar el tipo de paquete a regalos de Navidad (HR0 = 0)
3. Habilitar la verificación de inventario (C10 = Verdadero)
4. Habilitar el registro de auditoría (C13 = Verdadero)
5. Verificar que C15 nunca se haya activado

El pedido es crítico. Si cambiamos HR0 antes de desactivar C11, la trampa se activa.

Crear el script de remediación:

Terminal

```
root@tryhackme:~# nano restore_christmas.py
```

Introduzca el siguiente código:

```
#!/usr/bin/env python3
from pymodbus.client import ModbusTcpClient
import time

PLC_IP = "MACHINE_IP"
PORT = 502
UNIT_ID = 1

def read_coil(client, address):
    result = client.read_coils(address=address, count=1, slave=UNIT_ID)
    if not result.isError():
        return result.bits[0]
    return None
```

```
def read_register(client, address):
    result = client.read_holding_registers(address=address, count=1, slave=UNIT_ID)
    if not result.isError():
        return result.registers[0]
    return None

# Connect to PLC
client = ModbusTcpClient(PLC_IP, port=PORT)

if not client.connect():
    print("Failed to connect to PLC")
    exit(1)

print("=" * 60)
print("TBFC Drone System - Christmas Restoration")
print("=" * 60)
print()

# Step 1: Check current state
print("Step 1: Verifying current system state...")
time.sleep(1)

package_type = read_register(client, 0)
protection = read_coil(client, 11)
armed = read_coil(client, 15)

print(f" Package Type: {package_type} (1 = Eggs)")
print(f" Protection Active: {protection}")
print(f" Self-Destruct Armed: {armed}")
print()

# Step 2: Disable protection
print("Step 2: Disabling protection mechanism...")
time.sleep(1)

result = client.write_coil(11, False, slave=UNIT_ID)
if not result.isError():
    print(" Protection DISABLED")
    print(" Safe to proceed with changes")
else:
    print(" FAILED to disable protection")
    client.close()
    exit(1)

print()
time.sleep(1)

# Step 3: Change package type to Christmas
print("Step 3: Setting package type to Christmas presents...")
time.sleep(1)

result = client.write_register(0, 0, slave=UNIT_ID)
if not result.isError():
    print(" Package type changed to: Christmas Presents")
else:
    print(" FAILED to change package type")

print()
time.sleep(1)

# Step 4: Enable inventory verification
print("Step 4: Enabling inventory verification...")
time.sleep(1)

result = client.write_coil(10, True, slave=UNIT_ID)
if not result.isError():
    print(" Inventory verification ENABLED")
else:
    print(" FAILED to enable verification")

print()
time.sleep(1)

# Step 5: Enable audit logging
print("Step 5: Enabling audit logging...")
time.sleep(1)

result = client.write_coil(13, True, slave=UNIT_ID)
if not result.isError():
    print(" Audit logging ENABLED")
    print(" Future changes will be logged")
else:
    print(" FAILED to enable logging")

print()
time.sleep(2)

# Step 6: Verify restoration
print("Step 6: Verifying system restoration...")
time.sleep(1)

christmas_restored = read_coil(client, 14)
```

```
new_package_type = read_register(client, 0)
emergency_dump = read_coil(client, 12)
self_destruct = read_coil(client, 15)

print(f" Package Type: {new_package_type} (0 = Christmas)")
print(f" Christmas Restored: {christmas_restored}")
print(f" Emergency Dump: {emergency_dump}")
print(f" Self-Destruct Armed: {self_destruct}")
print()

if christmas_restored and new_package_type == 0 and not emergency_dump and not self_destruct:
    print("=" * 60)
    print("SUCCESS - CHRISTMAS IS SAVED")
    print("=" * 60)
    print()
    print("Christmas deliveries have been restored")
    print("The drones will now deliver presents, not eggs")
    print("Check the CCTV feed to see the results")
    print()

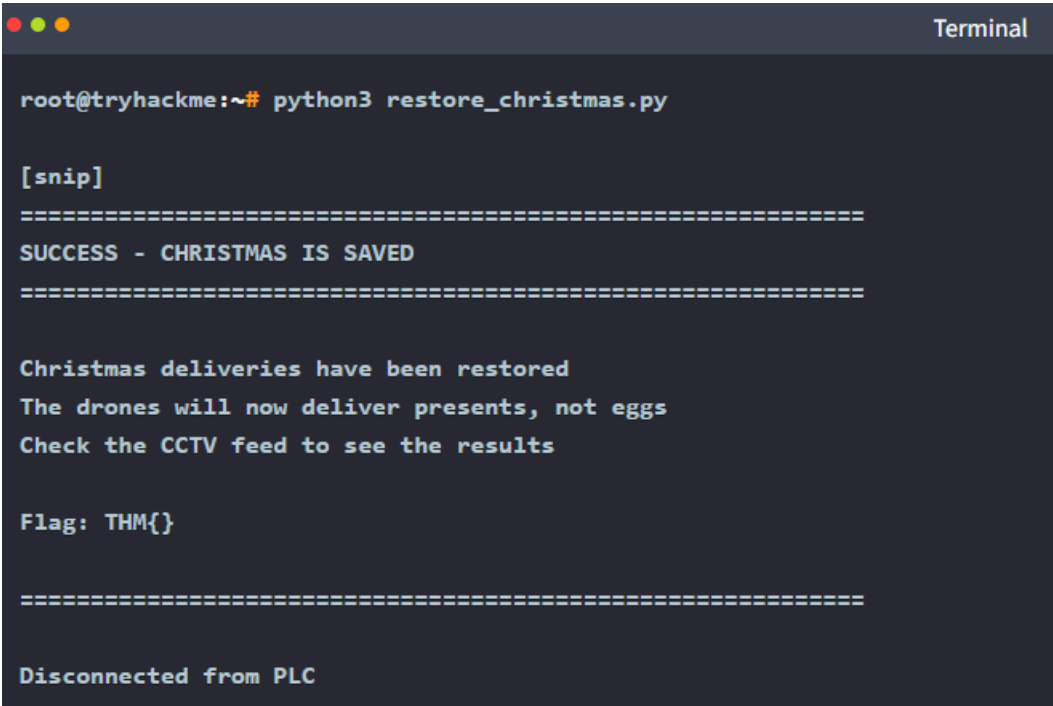
    # Read the flag from registers
    flag_result = client.read_holding_registers(address=20, count=12, slave=UNIT_ID)
    if not flag_result.isError():
        flag_bytes = []
        for reg in flag_result.registers:
            flag_bytes.append(reg >> 8)
            flag_bytes.append(reg & 0xFF)
        flag = ''.join(chr(b) for b in flag_bytes if b != 0)
        print(f"Flag: {flag}")

    print()
    print("=" * 60)

else:
    print("Restoration incomplete - check system state")

client.close()
print()
print("Disconnected from PLC")
```

Guardar y salir. Ejecutar el script de restauración:



¡Excelente trabajo! Ahora revisa la señal de CCTV en http://MACHINE_IP; deberías ver la derrota del Rey Malhare.

¿Qué pasaría si hubieras activado la trampa?

Si hubieras intentado cambiar HR0 antes de desactivar C11, esto habría sucedido:

- C15 (Autodestrucción) se activaría inmediatamente.
- Comenzaría una cuenta regresiva de 30 segundos.
- Después de 30 segundos, se activaría C12 (Descarga de emergencia).
- HR1 cambiaría a 10 (zona oceánica).
- Se descargaría todo el inventario restante.
- El CCTV mostraría la pantalla de activación de la trampa.
- Tendrías que reiniciar el desafío.

Esto demuestra por qué es fundamental comprender los sistemas de control industrial antes de realizar cambios. En situaciones reales, activar mecanismos de seguridad o trampas podría tener graves consecuencias físicas.

Análisis posterior al incidente

El ataque de King Malhare fue sofisticado porque:

- Utilizó acceso Modbus no autenticado (puerto 502);
- Manipuló valores de configuración directamente a nivel de protocolo;
- Desactivó mecanismos de seguridad (verificación, registro);
- Implementó una trampa para evitar una fácil remediación;

- Dejó una firma (666) como tarjeta de visita.

El técnico de mantenimiento que dejó la nota probablemente descubrió la vulnerabilidad, pero fue interrumpido antes de poder solucionarla. Su documentación salvó la Navidad al advertir sobre el mecanismo de la trampa.

¡Felicitaciones! Ha investigado y remediado con éxito una vulnerabilidad en un sistema de control industrial. Ha aprendido cómo funcionan los sistemas SCADA, los PLC, la comunicación Modbus y, lo más importante, cómo los atacantes pueden manipular estos sistemas y cómo restaurarlos de forma segura.

Answer the questions below

What's the flag?

Respuesta: THM{eGgMas0V3r}

```
root@ip-10-81-89-98: ~
File Edit View Search Terminal Help
Step 5: Enabling audit logging...
  Audit logging ENABLED
  Future changes will be logged

Step 6: Verifying system restoration...
  Package Type: 0 (0 = Christmas)
  Christmas Restored: True
  Emergency Dump: False
  Self-Destruct Armed: False

=====
SUCCESS - CHRISTMAS IS SAVED
=====

Christmas deliveries have been restored
The drones will now deliver presents, not eggs
Check the CCTV feed to see the results

Flag: THM{eGgMas0V3r}

=====

Disconnected from PLC
root@ip-10-81-89-98:~#
```

If you enjoyed today's room, feel free to check out our OT challenge: [Industrial Intrusion](#)

No answer needed